BỘ KHOA HỌC VÀ CÔNG NGHỆ

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

BỘ MÔN LẬP TRÌNH MẠNG

BÁO CÁO BÀI TẬP LỚN

GAME ĐẶT BOMB MULTIPLAYER

Người hướng dẫn : MSc. Ngô Văn Quyền

Nhóm : 05

Họ và tên sinh viên : Trịnh Trọng Chinh – B23DCCN097

Nguyễn Phạm Hoàng Duy – B23DCCN246

Nguyễn Trường Giang – B23DCCN257

Đào Xuân Mai – B23DCCN523

Chuyên ngành : Công nghệ phần mềm

Khoa : Công nghệ thông tin

Hà Nội, 2026

LỜI CẢM ƠN

Nhóm xin cảm ơn thầy Ngô Văn Quyền đã cung cấp nền tảng kiến thức về lập trình mạng. Những kiến thức này là cơ sở để nhóm xây dựng trò chơi Game Đặt Bomb Multiplayer và tổ chức việc kiểm tra hoạt động của hệ thống.

Nhóm cũng trân trọng các tác giả của những dự án mã nguồn mở và tài liệu kỹ thuật được tham khảo trong báo cáo. Việc tiếp cận các cách triển khai khác nhau giúp nhóm xác định rõ hơn các yêu cầu về đồng bộ trạng thái, quản lý kết nối và đánh giá hệ thống.

Báo cáo còn giới hạn ở quy mô thử nghiệm và môi trường triển khai. Nhóm mong nhận được góp ý để tiếp tục hoàn thiện sản phẩm và phương pháp đánh giá.

MỤC LỤC

LỜI CẢM ƠN **2**
GAME ĐẶT BOMB MULTIPLAYER **4**
TÓM TẮT **4**
I. GIỚI THIỆU **5**
II. CÁC CÔNG TRÌNH LIÊN QUAN **6**
2.1. DynaDungeons 6
2.2. Bomberman Battle 6
2.3. Bomberman của shuoros 7
2.4. Tổng hợp và đánh giá hạn chế 7
III. PHƯƠNG PHÁP VÀ THIẾT KẾ HỆ THỐNG **8**
3.1. Phương pháp tiếp cận và phân tích bài toán 8
3.2. Thiết kế kiến trúc và giao tiếp mạng 9
3.3. Phương pháp đồng bộ trạng thái 10
3.4. Xử lý gameplay và khôi phục kết nối 13
3.5. Thiết kế và triển khai hệ thống 15
3.6. Phương pháp kiểm thử và đánh giá 16
IV. KIỂM THỬ SƠ BỘ VÀ HOÀN THIỆN **18**
V. KẾT QUẢ VÀ THẢO LUẬN **19**
VI. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN **21**
TÀI LIỆU THAM KHẢO **22**
PHỤ LỤC **23**
Phụ lục A Yêu cầu và tiêu chí kiểm chứng 23
Phụ lục B Kịch bản kiểm thử chức năng 24
Phụ lục C Bảng Benchmark hiệu năng mạng 26
Phụ lục D Kiểm tra khôi phục kết nối 28

GAME ĐẶT BOMB MULTIPLAYER

TÓM TẮT

Đề tài giải quyết bài toán xây dựng Game Đặt Bomb Multiplayer với yêu cầu xử lý tương tác và đồng bộ trạng thái qua mạng. Hệ thống áp dụng mô hình client–server, trong đó máy chủ quản lý logic trò chơi và trao đổi dữ liệu qua TCP. Ứng dụng được triển khai bằng Java và Swing, hỗ trợ phòng chơi, chế độ game, trò chuyện, di chuyển, đặt bom, vật phẩm và kết quả trận đấu. Hồ sơ kiểm chứng ghi nhận 333 bài kiểm thử đạt cùng chín kịch bản đo loopback; khả năng kết nối lại được bổ sung trong bản cập nhật sau đó. Hệ thống tích hợp các chức năng gameplay, quản lý phòng chơi, đồng bộ trạng thái và xử lý kết nối, đồng thời được kiểm thử và đánh giá nhằm xác định mức độ đáp ứng các yêu cầu đặt ra.

Từ khóa: Bomb It; trò chơi nhiều người chơi; client–server; TCP; Java Swing; đồng bộ trạng thái; lập trình mạng.

I. GIỚI THIỆU

Trong các trò chơi multiplayer, trải nghiệm phụ thuộc không chỉ vào luật chơi và giao diện mà còn vào khả năng phối hợp giữa các máy qua mạng. Với trò chơi đặt bom một thao tác di chuyển hoặc đặt bom có thể ảnh hưởng ngay đến người chơi khác. Nếu trạng thái không nhất quán, các máy khách có thể hiển thị vị trí, vụ nổ hoặc kết quả khác nhau. Vì vậy, bài toán xây dựng trò chơi này tạo điều kiện vận dụng kiến thức lập trình mạng vào một ứng dụng có tương tác thời gian thực.

Khó khăn chính nằm ở việc tổ chức xử lý đồng thời mà vẫn duy trì một trạng thái trận đấu đáng tin cậy: một yêu cầu đặt bom có thể đến khi người chơi đang đổi ô; nhiều vụ nổ có thể cùng tác động lên một nhân vật; người chơi cũng có thể mất kết nối giữa trận. Nếu từng máy tự quyết định các tình huống này, kết quả dễ khác nhau. Động lực của đề tài là xây dựng một hệ thống có quy tắc xử lý rõ ràng và có bằng chứng kiểm tra được.

Đề tài phát triển Game Đặt Bomb Multiplayer theo mô hình client–server, sử dụng Java, giao diện Swing và TCP. Máy khách gửi ý định thao tác, còn máy chủ xử lý di chuyển, va chạm, bom, vật phẩm và kết quả. Các trạng thái được truyền về để máy khách hiển thị và hiệu chỉnh dự đoán cục bộ. Mục tiêu là hoàn thiện một vòng đời chơi gồm kết nối, tạo hoặc tham gia phòng, sẵn sàng, bắt đầu, chơi, nhận kết quả và chơi lại.

Ba câu hỏi định hướng cho quá trình thực hiện là:

RQ1: Làm thế nào để xây dựng trò chơi đặt bom nhiều người chơi theo mô hình client–server, bảo đảm các thao tác di chuyển, đặt bom, nhặt vật phẩm và xử lý sát thương được thực hiện thống nhất khi nhiều người chơi tương tác đồng thời?

RQ2: Làm thế nào để đồng bộ trạng thái trò chơi giữa máy chủ và các máy khách, đồng thời hạn chế ảnh hưởng của độ trễ mạng đến khả năng điều khiển và hiển thị chuyển động của nhân vật?

RQ3: Làm thế nào để bảo đảm khả năng chịu lỗi và duy trì hoạt động ổn định của trò chơi nhiều người chơi khi xảy ra các sự cố trong quá trình giao tiếp mạng?

Phạm vi đề tài tập trung vào việc xây dựng trò chơi Game Đặt Bomb Multiplayer trên nền tảng máy tính, hỗ trợ từ 2–4 người chơi tham gia cùng một trận đấu thông qua mạng. Phạm vi chức năng bao gồm quản lý phòng chơi, trò chuyện, lựa chọn chế độ, lựa chọn hình thức thi đấu, điều khiển nhân vật, đặt bom, vật phẩm, sát thương và xác định kết quả. Về mặt kỹ thuật, đề tài tập trung giải quyết các vấn đề xử lý tương tác đồng thời, đồng bộ trạng thái giữa các máy khách, hạn chế ảnh hưởng của độ trễ và xử lý sự cố kết nối trong quá trình chơi. Hệ thống được triển khai và kiểm thử bước đầu trên môi trường Windows, chưa hướng đến quy mô thương mại. Kết quả của đề tài là một trò chơi multiplayer tích hợp các cơ chế xử lý và đồng bộ qua mạng, đồng thời được kiểm thử, đánh giá nhằm xác định mức độ đáp ứng các yêu cầu đặt ra.

II. CÁC CÔNG TRÌNH LIÊN QUAN

Các trò chơi thuộc thể loại đặt bom chiến thuật đã được triển khai trên nhiều nền tảng với những hình thức chơi khác nhau, từ nhiều người chơi trên cùng thiết bị đến kết nối giữa các máy tính qua mạng. Trong phạm vi đề tài, nhóm khảo sát ba dự án mã nguồn mở gồm DynaDungeons, Bomberman Battle và Bomberman của shuoros nhằm tìm hiểu cách tổ chức gameplay, công nghệ sử dụng và những vấn đề liên quan đến phát triển trò chơi nhiều người chơi.

2.1. DynaDungeons

DynaDungeons là dự án được phát triển bằng Godot, hỗ trợ tối đa bốn người chơi trên cùng một thiết bị. Trò chơi triển khai các cơ chế đặc trưng của Bomberman như di chuyển trên bản đồ, đặt bom, phá vật cản và thu thập vật phẩm tăng sức mạnh. Dự án cung cấp hướng tham khảo về thiết kế luật chơi và tổ chức tương tác giữa các nhân vật. Tuy nhiên, do tập trung vào multiplayer cục bộ, dự án không trực tiếp giải quyết những vấn đề đặc trưng của trò chơi qua mạng như truyền thông giữa các máy tính, độ trễ và đồng bộ trạng thái phân tán.

2.2. Bomberman Battle

Bomberman Battle mở rộng hình thức chơi sang môi trường mạng, hỗ trợ tối đa bốn người chơi, chế độ hai người dùng chung bàn phím và đối thủ máy. Dự án cho phép

tổ chức kết nối theo mô hình P2P hoặc client–server, trong đó người chơi có thể tham gia thông qua địa chỉ IP. Đây là hướng triển khai gần với mục tiêu xây dựng trò chơi multiplayer của đề tài. Tuy nhiên, tác giả cho biết người chơi sử dụng các nhà cung cấp dịch vụ Internet khác nhau có thể gặp khó khăn hoặc không thể thiết lập kết nối. Hạn chế này cho thấy khả năng kết nối qua Internet của trò chơi còn phụ thuộc vào điều kiện và cấu hình mạng, chưa bảo đảm hoạt động trong mọi môi trường. Bên cạnh đó, khả năng duy trì và khôi phục phiên chơi khi xảy ra gián đoạn kết nối mạng chưa được làm rõ trong phạm vi dự án.

2.3. Bomberman của shuoros

Bomberman của shuoros có nhiều điểm tương đồng với đề tài về công nghệ triển khai khi sử dụng Java, Swing và WebSockets. Dự án hỗ trợ chơi trực tuyến, tạo phòng tùy chỉnh và các cơ chế gameplay như đặt bom, phá vật cản. Việc sử dụng Java và giao tiếp mạng giúp dự án trở thành một trường hợp tham khảo phù hợp về tổ chức ứng dụng multiplayer. Tuy nhiên, khả năng duy trì tính nhất quán của trạng thái trò chơi khi xảy ra độ trễ mạng, cũng như cơ chế xử lý và khôi phục phiên chơi khi kết nối bị gián đoạn, vẫn là những khía cạnh cần được làm rõ và đánh giá thêm.

2.4. Tổng hợp và đánh giá hạn chế

Qua khảo sát, có thể thấy các dự án trên đã triển khai những chức năng cơ bản của trò chơi Bomberman theo các hướng khác nhau. DynaDungeons tập trung vào gameplay nhiều người chơi cục bộ, trong khi Bomberman Battle và Bomberman của shuoros mở rộng khả năng chơi qua mạng. Tuy nhiên, việc bảo đảm hiệu quả đồng bộ trạng thái, hạn chế ảnh hưởng của độ trễ mạng và duy trì phiên chơi khi xảy ra gián đoạn kết nối vẫn là những vấn đề kỹ thuật cần được quan tâm và đánh giá thêm. Trên cơ sở đó, đề tài Game Đặt Bomb Multiplayer tập trung xây dựng trò chơi nhiều người chơi bằng Java theo kiến trúc client–server, sử dụng TCP để trao đổi dữ liệu và máy chủ làm thành phần quyết định trạng thái trò chơi. Hệ thống kết hợp Full Snapshot và Delta Snapshot để đồng bộ trạng thái, Client-side Prediction và Reconciliation để xử lý độ trễ điều khiển, cùng Interpolation để cải thiện hiển thị

chuyển động, triển khai cơ chế khôi phục kết nối khi mất kết nối, người chơi có 30 giây để tự động kết nối lại và đồng bộ trạng thái bằng Full Snapshot, nếu quá hạn sẽ bị loại khỏi trận đấu. Ngoài ra, đề tài xây dựng các kịch bản kiểm thử, đo lường hoạt động mạng.

Điểm khác biệt mà đề tài hướng đến không nằm ở việc đề xuất một thuật toán đồng bộ mới, mà ở việc kết hợp các kỹ thuật mạng trong một hệ thống hoàn chỉnh, mô tả rõ phương pháp triển khai và đánh giá bằng những chỉ số có thể kiểm chứng. Các nhận xét về hạn chế của công trình liên quan được giới hạn trong phạm vi thông tin công khai đã khảo sát, không đồng nghĩa với việc các hệ thống đó hoàn toàn không có những cơ chế tương ứng.

III. PHƯƠNG PHÁP VÀ THIẾT KẾ HỆ THỐNG

3.1. Phương pháp tiếp cận và phân tích bài toán

Đề tài Game Đặt Bomb Multiplayer được thực hiện theo hướng phân tích yêu cầu, thiết kế giải pháp, triển khai hệ thống và kiểm thử thực nghiệm. Trọng tâm là giải quyết các vấn đề phát sinh khi nhiều người chơi cùng tương tác trong một môi trường trò chơi thông qua mạng máy tính.

Trong trò chơi đặt bom nhiều người chơi, các thao tác như di chuyển, đặt bom, nhặt vật phẩm và chịu sát thương có thể diễn ra gần như đồng thời. Do ảnh hưởng của độ trễ mạng và thời điểm xử lý thông điệp, trạng thái hiển thị giữa các máy khách có nguy cơ không đồng nhất. Bên cạnh đó, việc truyền dữ liệu liên tục có thể tạo ra lưu lượng dư thừa, trong khi gián đoạn kết nối có thể ảnh hưởng đến quá trình tham gia trận đấu.

Từ những đặc điểm trên, nhóm xác định bốn vấn đề kỹ thuật chính cần giải quyết:

- **Xử lý đồng thời:** Bảo đảm các thao tác từ nhiều máy khách được xử lý thống nhất, tránh tranh chấp khi cập nhật trạng thái trò chơi.
- **Đồng bộ trạng thái:** Duy trì tính nhất quán giữa máy chủ và các máy khách, đồng thời hạn chế ảnh hưởng của độ trễ đến hiển thị và điều khiển.
- **Tối ưu truyền dữ liệu:** Giảm lượng dữ liệu không cần thiết trong quá trình cập nhật trạng thái trò chơi.

- **Quản lý kết nối:** Phát hiện và xử lý gián đoạn kết nối, hỗ trợ người chơi tiếp tục trận trong điều kiện cho phép.

Để giải quyết các vấn đề này, nhóm sử dụng kiến trúc client–server với máy chủ quản lý trạng thái chính thức, kết hợp cơ chế xử lý lệnh tập trung, đồng bộ Full/Delta Snapshot, dự đoán và hiệu chỉnh trạng thái tại máy khách, cùng cơ chế khôi phục kết nối có giới hạn thời gian.

Bên cạnh các yêu cầu đồng bộ thông thường, hai chế độ Shadow Relay và Paradox đặt ra những ràng buộc bổ sung. Trong Shadow Relay, các đội có phạm vi quan sát khác nhau, vì vậy máy chủ phải phân phối dữ liệu theo quyền quan sát của từng người nhận. Trong Paradox, thao tác quay ngược trạng thái gameplay làm thay đổi lịch sử mô phỏng, đòi hỏi hệ thống phân biệt các timeline và ngăn dữ liệu thuộc timeline cũ tiếp tục tác động đến trận đấu.

Các giải pháp được kiểm chứng thông qua kiểm thử chức năng và đo mạng. Việc đánh giá tập trung vào tính đúng đắn của các cơ chế đã triển khai và đặc điểm hoạt động của hệ thống trong những kịch bản thử nghiệm cụ thể.

3.2. Thiết kế kiến trúc và giao tiếp mạng

Hệ thống được xây dựng theo mô hình client–server, sử dụng Java và giao thức TCP để trao đổi dữ liệu. Máy chủ chịu trách nhiệm quản lý phiên kết nối, phòng chơi, xử lý gameplay và duy trì trạng thái chính thức của trận đấu. Máy khách tiếp nhận thao tác người dùng, gửi yêu cầu đến máy chủ và hiển thị trạng thái nhận được thông qua giao diện Swing.

Một vấn đề khi sử dụng TCP là dữ liệu được truyền dưới dạng luồng byte, không bảo toàn ranh giới giữa các thông điệp ứng dụng. Vì vậy, nhóm thiết kế cơ chế đóng khung thông điệp theo cấu trúc:

[length:int32][type:int16][payload:UTF-8 JSON]

Trong đó, length biểu thị độ dài của phần type và payload, type xác định loại thông điệp và payload chứa dữ liệu tương ứng. Bộ giải mã xử lý được trường hợp một thông điệp bị chia qua nhiều lần đọc hoặc nhiều thông điệp xuất hiện trong cùng một lần đọc. Độ dài frame hợp lệ được giới hạn từ 2 đến 1.048.576 byte.

Để quản lý tính hợp lệ của thông điệp, giao thức sử dụng protocolVersion trong quá trình handshake, matchId để phân biệt các trận đấu và số thứ tự hoặc định danh yêu cầu đối với những thao tác cần xác nhận. Máy chủ xác định danh tính người gửi từ phiên kết nối đã được xác nhận, thay vì tin trực tiếp thông tin gameplay do máy khách cung cấp.

Đối với vấn đề xử lý đồng thời, nếu các luồng mạng trực tiếp thay đổi GameState, nhiều thao tác có thể tranh chấp cùng một trạng thái và tạo ra kết quả phụ thuộc vào thứ tự thực thi luồng. Để hạn chế vấn đề này, nhóm sử dụng Command Queue kết hợp Game Loop.

Hệ thống quản lý các chế độ chơi qua registry phía máy chủ, kiểm tra số người chơi, phiên bản luật và yêu cầu máy khách trước khi bắt đầu trận. Các chế độ dùng chung Game Loop và giao tiếp mạng, còn luật riêng được triển khai qua factory và gameplay extension, giúp dễ dàng mở rộng và tái sử dụng các cơ chế hiện có.

Các thông điệp gameplay sau khi được kiểm tra sẽ được đưa vào Command Queue. Game Loop tiếp nhận, xử lý các lệnh và cập nhật trạng thái trò chơi theo quy tắc thống nhất. Luồng xử lý chính được tổ chức như sau:

Client → TCP → Dispatcher → Command Queue → Game Loop → GameState → Snapshot/Event → Client.

Cách tổ chức này tách biệt quá trình giao tiếp mạng với xử lý gameplay, đồng thời bảo đảm các thay đổi trạng thái có thẩm quyền được thực hiện tập trung tại máy chủ.

3.3. Phương pháp đồng bộ trạng thái

3.3.1. Đồng bộ Full Snapshot và Delta Snapshot

Trong trò chơi nhiều người chơi, máy chủ cần liên tục cập nhật trạng thái nhân vật, bom, vật phẩm và bản đồ cho các máy khách. Nếu truyền toàn bộ trạng thái sau mỗi lần cập nhật, hệ thống có thể gửi lặp lại nhiều dữ liệu không thay đổi, làm tăng lưu lượng truyền thông.

Để giải quyết vấn đề này, nhóm kết hợp hai cơ chế Full Snapshot và Delta Snapshot.

Full Snapshot chứa trạng thái đầy đủ cần thiết để máy khách khởi tạo hoặc khôi phục trạng thái trò chơi. Trong khi đó, Delta Snapshot chỉ chứa những thay đổi so với trạng thái cơ sở đã biết, giúp hạn chế dữ liệu dư thừa trong các lần cập nhật thông thường. Về nguyên tắc, bản cập nhật chênh lệch giữa trạng thái tại tick b và tick t được xác định:

$$D_{b\to t} = \text{diff}(S_b, S_t)$$

Trạng thái mới được tái tạo bằng cách áp dụng bản chênh lệch lên trạng thái cơ sở:

$$S_t = \text{apply}(S_b, D_{b\to t})$$

Trong đó, S_b là trạng thái cơ sở, $S_t$là trạng thái cần tái tạo và $D_{b\to t}$ là tập hợp những thay đổi giữa hai trạng thái.

Việc sử dụng Delta Snapshot yêu cầu máy khách phải có baseline phù hợp. Nếu nhận được bản cập nhật không tương thích hoặc không thể tái tạo trạng thái, máy khách không áp dụng delta mà yêu cầu Full Snapshot để đồng bộ lại. Phía máy chủ chỉ cập nhật baseline sau khi bản cập nhật đã được đưa thành công vào hàng đợi truyền. Khi khôi phục kết nối hoặc chuyển timeline trong Paradox, baseline được thiết lập lại và máy khách cần nhận Full Snapshot hợp lệ trước khi tiếp tục áp dụng delta.

Cách kết hợp này vừa hỗ trợ giảm dữ liệu truyền trong quá trình chơi, vừa duy trì khả năng khôi phục trạng thái khi cần thiết.

3.3.2. Client-side Prediction và Reconciliation

Một vấn đề khác của trò chơi qua mạng là độ trễ giữa thời điểm người chơi thực hiện thao tác và thời điểm máy chủ phản hồi. Nếu máy khách chỉ cập nhật vị trí sau khi nhận được trạng thái từ máy chủ, nhân vật có thể phản ứng chậm so với thao tác điều khiển.

Để hạn chế hiện tượng này, nhóm sử dụng Client-side Prediction. Khi người chơi thực hiện thao tác di chuyển, máy khách dự đoán và cập nhật vị trí cục bộ, đồng thời gửi input kèm số thứ tự đến máy chủ.

Do vị trí dự đoán có thể khác với kết quả xử lý chính thức, nhóm kết hợp thêm Reconciliation để hiệu chỉnh trạng thái. Khi nhận snapshot từ máy chủ, máy khách

thực hiện ba thao tác: khôi phục trạng thái có thẩm quyền, loại bỏ những input đã được xác nhận và áp dụng lại các input còn chờ xử lý.

Nhờ đó, thao tác điều khiển có thể được phản hồi sớm hơn trên máy khách, trong khi máy chủ vẫn giữ quyền quyết định vị trí và kết quả gameplay cuối cùng.

3.3.3. Interpolation

Đối với các nhân vật khác, việc hiển thị trực tiếp vị trí tại từng snapshot có thể khiến chuyển động bị giật khi các bản cập nhật được nhận không đều.

Nhóm sử dụng Interpolation để nội suy vị trí giữa hai trạng thái đã nhận. Giả sử nhân vật có vị trí p_a tại tick a và p_b tại tick b, vị trí hiển thị tại tick r được tính như sau:

$$\alpha = \text{clamp}\left(\frac{r-a}{b-a}, 0{,}1\right)$$

$$p(r) = (1-\alpha)p_a + \alpha p_b$$

Phương pháp này giúp chuyển động của các nhân vật khác liên tục hơn trên màn hình. Interpolation chỉ phục vụ hiển thị, không thay đổi trạng thái gameplay do máy chủ quyết định.

Như vậy, ba nhóm cơ chế được kết hợp để giải quyết các vấn đề khác nhau: Full/Delta Snapshot phục vụ đồng bộ và giảm dữ liệu truyền, Prediction kết hợp Reconciliation hỗ trợ phản hồi điều khiển, còn Interpolation cải thiện tính liên tục của chuyển động.

3.3.4. Đồng bộ trạng thái trong chế độ Shadow Relay

Trong chế độ Shadow Relay, hệ thống cần đồng bộ thêm trạng thái trạm, điểm của hai đội và thông tin quét. Trạng thái trạm bao gồm đội sở hữu, tiến độ chiếm và tình trạng tranh chấp. Máy chủ xử lý các thay đổi này trong vòng lặp trò chơi và gửi kết quả về máy khách để hiển thị thống nhất.

Bên cạnh đó, chế độ này sử dụng cơ chế giới hạn tầm nhìn và quét để hỗ trợ quan sát. Thông tin hiển thị cần tuân theo trạng thái quan sát do máy chủ xác định. Vì vậy, việc đồng bộ không chỉ bảo đảm tính nhất quán của trạng thái trận đấu mà còn phải phù hợp với phạm vi thông tin mà người nhận được phép quan sát.

3.3.5. Đồng bộ trạng thái khi đảo thời gian trong chế độ Paradox

Chế độ Paradox cho phép khôi phục trạng thái trò chơi về hai giây trước. Để thực hiện chức năng này, máy chủ lưu lịch sử trạng thái mô phỏng dưới dạng các checkpoint. Khi nhận yêu cầu đảo thời gian, máy chủ kiểm tra điều kiện sử dụng và lịch sử hiện có trước khi khôi phục checkpoint tương ứng.

Việc đảo thời gian tạo ra yêu cầu phân biệt trạng thái của nhánh thời gian cũ với nhánh hiện tại. Hệ thống sử dụng định danh nhánh thời gian để kiểm tra các yêu cầu liên quan, tránh việc thao tác thuộc nhánh cũ tiếp tục tác động lên trạng thái đã phục hồi. Sau khi khôi phục, phần lịch sử thuộc tương lai cũ được loại bỏ và quá trình mô phỏng tiếp tục từ checkpoint đã chọn.

Thời gian mô phỏng được quản lý riêng với thứ tự cập nhật phục vụ truyền tải. Lượt sử dụng kỹ năng cũng không được hoàn lại khi đảo thời gian. Cách tổ chức này giúp tránh nhầm lẫn giữa việc phục hồi trạng thái gameplay và việc xử lý lại các thông điệp mạng.

3.4. Xử lý gameplay và khôi phục kết nối

3.4.1. Xử lý các tương tác gameplay

Trong Game Đặt Bomb Multiplayer, nhiều thao tác có thể tác động đến cùng một đối tượng, chẳng hạn hai người chơi cùng nhặt vật phẩm hoặc nhiều vụ nổ đồng thời gây sát thương lên một nhân vật. Nếu kết quả được quyết định riêng tại từng máy khách, trạng thái trò chơi có thể không nhất quán.

Để giải quyết vấn đề này, nhóm áp dụng cơ chế server-authoritative, trong đó máy chủ kiểm tra và quyết định kết quả của các tương tác gameplay.

Đối với di chuyển, máy chủ xử lý ý định điều khiển và kiểm tra va chạm với bản đồ. Đối với đặt bom, máy chủ kiểm tra điều kiện đặt, số bom hiện có và vị trí hợp lệ. Yêu cầu đặt bom sử dụng requestId để tránh tạo thêm bom khi cùng một yêu cầu được gửi lặp. Đối với vật phẩm, máy chủ xác định người nhặt và chỉ áp dụng hiệu ứng một lần. Các vụ nổ có thể kích hoạt dây chuyền, nhưng sát thương lên một nhân vật trong cùng

đợt được tính theo quy tắc thống nhất, tránh gây sát thương lặp. Việc sử dụng khiên, giảm mạng, hồi sinh và xác định kết quả trận đấu cũng được xử lý tại máy chủ.
Nhờ đó, các máy khách nhận cùng kết quả gameplay chính thức thay vì tự đưa ra quyết định độc lập.

3.4.2. Khôi phục kết nối

Trong quá trình chơi, người dùng có thể mất kết nối tạm thời. Nếu loại người chơi ngay khi phát hiện gián đoạn, một sự cố mạng ngắn có thể khiến họ không thể tiếp tục trận đấu. Ngược lại, việc giữ phiên không giới hạn có thể gây lãng phí tài nguyên. Nhóm lựa chọn cơ chế giữ phiên có thời hạn. Theo thiết kế hiện hành, máy chủ giữ chỗ cho người chơi trong 30 giây kể từ khi phát hiện mất kết nối. Trong thời gian này, trận đấu vẫn tiếp tục; nhân vật mất kết nối ngừng di chuyển nhưng vẫn chịu tác động từ bom và các sự kiện gameplay.
Khi ứng dụng máy khách còn hoạt động, hệ thống thử khôi phục phiên bằng định danh và token đang lưu trong bộ nhớ. Sau khi kết nối được xác nhận, máy khách nhận Full Snapshot để đồng bộ trạng thái trước khi tiếp tục gửi thao tác gameplay.
Máy chủ cũng kiểm tra thế hệ kết nối để tránh callback hoặc timer thuộc socket cũ tác động nhầm lên kết nối mới. Nếu thời gian giữ chỗ kết thúc mà người chơi chưa quay lại, hệ thống xử lý loại người chơi mất kết nối theo luật.
Cơ chế này hỗ trợ khôi phục những gián đoạn tạm thời nhưng không tự khởi động lại ứng dụng đã đóng và không áp dụng cho tình huống chủ động thoát trận.

3.3.4. Các chế độ chơi

1. Classic: Người chơi đặt bom phá vật cản, gây sát thương, thu thập vật phẩm và sử dụng cơ chế mạng, hồi sinh. Máy chủ xử lý thao tác, vụ nổ và xác định kết quả. Chế độ hỗ trợ thi đấu cá nhân hoặc đội 2v2.
2. Shadow Relay: Chế độ 2v2, trong đó hai đội tranh quyền kiểm soát trạm để tích lũy điểm. Trạm bị tranh chấp khi cả hai đội cùng hiện diện. Đội đạt 60 điểm, có điểm cao hơn khi hết giờ hoặc loại được đối phương sẽ chiến thắng. Chế độ kết hợp giới hạn tầm nhìn và bom quét (phím F), với thông tin quan sát do máy chủ kiểm soát.

3. Paradox: Cho phép người chơi đảo ngược trạng thái trận đấu về hai giây trước bằng phím R, mỗi người một lượt. Máy chủ kiểm tra điều kiện, phục hồi từ checkpoint và áp dụng thời gian chờ chung ba giây. Lịch sử tương lai bị loại bỏ, lượt đã sử dụng không được hoàn lại và kết quả trận đã chốt không thể đảo ngược.

3.5. Thiết kế và triển khai hệ thống

Từ các phương pháp đã lựa chọn, hệ thống được phân chia thành các thành phần chính gồm máy chủ, máy khách và thư viện giao thức dùng chung.

Thành phần	Chức năng
Client UI và Renderer	Nhận thao tác, hiển thị bản đồ, nhân vật và kết quả
ClientNetwork	Gửi nhận thông điệp, heartbeat và quản lý kết nối
Session và Room	Quản lý phiên, thành viên và trạng thái phòng
Dispatcher và Command Queue	Kiểm tra và điều phối các yêu cầu gameplay
GameLoop và GameState	Cập nhật trạng thái chính thức của trò chơi
Bomb và Item Systems	Xử lý bom, vụ nổ, vật phẩm, sát thương và hồi sinh
Snapshot và Codec	Mã hóa dữ liệu, đồng bộ Full/Delta và xử lý xác nhận

Các thao tác socket và xử lý dữ liệu mạng được tách khỏi luồng cập nhật giao diện nhằm hạn chế việc giao diện bị chặn bởi các tác vụ mạng.

Với cấu hình Classic mặc định, bản đồ có ô 32 × 32 pixel, nhân vật có hitbox 24 × 24 pixel và tốc độ ban đầu 80 pixel/giây. Bom nổ sau ba giây, mỗi người ban đầu đặt được một bom với tầm nổ một ô mỗi hướng, có ba mạng và thời lượng trận 180 giây. Hệ thống hỗ trợ năm loại vật phẩm, với xác suất xuất hiện khi phá vật cản mềm là 75%.

Shadow Relay yêu cầu bốn người chia hai đội, chiếm trạm để tích điểm và thắng khi đạt 60 điểm. Chế độ có cơ chế giới hạn tầm nhìn và bom quét bằng phím F. Paradox

hỗ trợ cá nhân 2–4 người hoặc đội 2–2, cho phép dùng phím R khôi phục trạng thái toàn bản đồ về hai giây trước, mỗi người một lượt.
Chủ phòng lựa chọn chế độ tại phòng chờ; máy chủ kiểm tra cấu hình, số người và xử lý luật chơi tương ứng.
Hệ thống hỗ trợ năm loại vật phẩm: tăng số bom, tăng tầm nổ, tăng tốc, thêm mạng và khiên. Khi vật cản mềm bị phá, vật phẩm có thể xuất hiện với xác suất mặc định 75%. Máy chủ quản lý việc nhặt vật phẩm, tính sát thương và xác định kết quả thắng hoặc hòa theo luật chơi.
Việc phân chia thành phần và tập trung xử lý gameplay tại máy chủ giúp hệ thống dễ tổ chức, kiểm thử và mở rộng chức năng trong phạm vi đề tài.

3.6. Phương pháp kiểm thử và đánh giá

Nhóm đánh giá hệ thống bằng hai phương pháp: kiểm thử chức năng tự động và đo hiệu năng giao tiếp mạng.

Kiểm thử chức năng được thực hiện bằng JUnit 5 và Maven. Các test gồm kiểm thử đơn vị, giao thức và tích hợp client–server. Nội dung bao phủ xử lý frame TCP, dữ liệu không hợp lệ, quyền trong phòng, di chuyển và va chạm, đặt bom lặp, nổ dây chuyền, tranh chấp vật phẩm, đồng bộ snapshot, yêu cầu resync và vòng đời trận đấu. Reconnect được kiểm tra riêng bằng các test về thời hạn giữ phiên, đồng bộ full state và gameplay trong thời gian client mất kết nối.

Đo hiệu năng mạng sử dụng NetworkBenchmark, kết nối client và server bằng TCP socket thật trên loopback. Server và runner chạy trên cùng máy; suite gồm 9 cấu hình thuộc các nhóm transport, idle, movement và bombs. Mỗi kịch bản có 2 giây warmup và cửa sổ đo khoảng 10 giây.

Benchmark thu thập RTT từ PING đến PONG, MOVE ACK từ lúc gửi input đến snapshot xác nhận sequence, TX/RX theo byte frame ứng dụng, số full/delta snapshot, resync request và trạng thái cleanup. P95 được tính riêng cho từng client theo nearest-rank; CSV lưu số liệu từng client, còn bảng tổng hợp trong báo cáo lấy

trung bình P95 giữa các client. Các chỉ số này đo thời gian quan sát ở runner, không phải thời gian xử lý nội bộ server.

Phép đo loopback không đại diện cho mạng LAN nhiều máy. Suite này cũng không đo trực tiếp trải nghiệm giao diện Swing, CPU/RAM hoặc khả năng vận hành dài hạn. Kết quả và giới hạn của từng phương pháp được trình bày ở phụ lục và chi tiết hơn ở src code.

IV. KIỂM THỬ SƠ BỘ VÀ HOÀN THIỆN

Nhóm sử dụng kiểm thử tự động và benchmark để đánh giá hệ thống trước khi nghiệm thu trên nhiều máy. Báo cáo Surefire ghi nhận 333 test case thuộc 76 test suite, tất cả đều đạt: 0 lỗi, 0 thất bại và 0 test bị bỏ qua. Mã production và test được biên dịch với release 25; môi trường chạy ghi nhận Java 25.0.4.1.

Các test bao phủ codec và xử lý frame TCP, dữ liệu không hợp lệ, quyền trong phòng, tạo và rời phòng, di chuyển và va chạm, đặt bom trùng yêu cầu, nổ dây chuyền, tranh chấp vật phẩm, kết quả trận, chơi lại, snapshot cũ và full resync. Kiểm thử độ ổn định còn xét hàng đợi đầy, peer đọc chậm, deadline ghi, heartbeat timeout, lỗi callback và giải phóng tài nguyên.

Trong nhóm reconnect, RecoveryGameplayIntegrationTest ghi nhận 8 test case tích hợp gameplay; SessionRecoveryTest ghi nhận 13 test case logic phiên, gồm kiểm tra biên thời hạn 15/30 giây, token sai, timer và socket cũ, reconnect đồng thời và lỗi khi thiết lập writer. Benchmark mạng được chạy như tiến trình Java riêng, nên kết quả không nằm trong bộ đếm JUnit. Phép đo dùng TCP loopback, server và runner trong cùng JVM; gồm 9 cấu hình transport, idle, movement và bombs, warmup 2 giây và cửa sổ đo khoảng 10 giây. CSV ghi nhận 25/25 dòng PASS, 1.238 mẫu RTT, cleanup_ok=true trên mọi dòng và không có yêu cầu resync. Đây là phép đo cục bộ, không đại diện cho LAN nhiều máy.

Các kết quả này xác nhận những hành vi được bao phủ bởi test và benchmark trong môi trường cục bộ. Việc gây gián đoạn mạng thực tế và kiểm tra giao diện Swing trên nhiều máy LAN vẫn cần được thực hiện để đánh giá trải nghiệm triển khai.

V. KẾT QUẢ VÀ THẢO LUẬN

5.1. Kết quả triển khai và câu hỏi RQ1

Đối với RQ1, đề tài đã xây dựng trò chơi đặt bom nhiều người chơi theo mô hình client–server, hỗ trợ phòng chơi từ 2–4 người. Hệ thống tích hợp các chức năng kết nối, tạo và tham gia phòng, điều khiển nhân vật, đặt bom, nhặt vật phẩm, xử lý sát thương, xác định kết quả và chơi lại.

Máy chủ chịu trách nhiệm quản lý trạng thái trận đấu và xử lý các thao tác thông qua Command Queue và Game Loop. Cơ chế này giúp bảo đảm các hành động được thực hiện thống nhất khi nhiều người chơi tương tác đồng thời. Kiểm thử thực nghiệm cho thấy hệ thống vận hành hoàn toàn ổn định với tần số xử lý lệnh chuẩn 20 Hz ở mức tải tối đa lên tới 122 client đồng thời (chia thành 31 phòng chơi), đạt tỷ lệ xử lý lệnh thành công tuyệt đối và duy trì tần số vòng lặp game ổn định quanh mức 20.14 Hz. Qua đó cho thấy chức năng cho thấy hệ thống đáp ứng các tình huống gameplay đã xây dựng, qua đó giải quyết RQ1 trong phạm vi đề tài.

5.2. Kết quả đồng bộ trạng thái và câu hỏi RQ2

Đối với RQ2, hệ thống áp dụng mô hình Server-Authoritative kết hợp Full Snapshot và Delta Snapshot để duy trì trạng thái thống nhất giữa các máy khách. Đồng thời, Client-side Prediction, Server Reconciliation và Interpolation được sử dụng nhằm hạn chế ảnh hưởng của độ trễ đến thao tác điều khiển và hiển thị chuyển động.

gian xử lý thao tác và cập nhật trạng thái, không chỉ là độ trễ truyền mạng.

Lưu lượng nhận tăng theo số lượng người chơi và mức độ hoạt động. Một bài kiểm tra riêng về Delta Snapshot cũng ghi nhận kích thước dữ liệu giảm từ 110.076 byte xuống 16.411 byte, tương đương 85,09% trong tình huống kiểm tra xác định. Những kết quả này bước đầu cho thấy hiệu quả của cơ chế đồng bộ.

5.3. Khả năng chịu lỗi và câu hỏi RQ3

Đối với RQ3, hệ thống triển khai các cơ chế xử lý sự cố kết nối nhằm duy trì hoạt động của trận đấu. Khi người chơi mất kết nối, máy chủ giữ phiên trong tối đa 30 giây để máy khách có thể tự động kết nối lại. Sau khi khôi phục, máy khách nhận Full Snapshot để đồng bộ trạng thái; nếu quá thời hạn, người chơi bị loại khỏi trận

đấu theo luật. Hệ thống cũng hỗ trợ đồng bộ lại khi Delta Snapshot không phù hợp với trạng thái tham chiếu, hạn chế nguy cơ sai lệch dữ liệu giữa các máy khách.

Năng lực khôi phục kết nối đã được tự động hóa và xác thực thông qua 21 kịch bản kiểm thử tích hợp. Các bài test này đã chứng minh hệ thống xử lý chính xác biên thời gian giới hạn 30 giây, cũng như khôi phục thành công trạng thái trò chơi khi xảy ra gián đoạn nhiều chu kỳ liên tiếp hoặc mất kết nối đồng loạt. Vì vậy, RQ3 đã được giải quyết hoàn thiện ở mức mô phỏng tự động và bảo đảm tính đúng đắn của mã nguồn.

5.4. Đóng góp và giới hạn của đề tài

Đề tài đã xây dựng trò chơi đặt bom multiplayer hỗ trợ 2–4 người, tích hợp các chức năng gameplay, xử lý tương tác đồng thời, đồng bộ trạng thái và khôi phục kết nối. Bên cạnh đó, hệ thống được kiểm thử chức năng và đo mạng bước đầu nhằm đánh giá khả năng đáp ứng các yêu cầu đặt ra.

Tuy nhiên, hệ thống vẫn còn một số hạn chế về kiến trúc và chức năng. Mỗi trận đấu hiện hỗ trợ tối đa bốn người chơi, chưa đáp ứng các trận đấu có quy mô lớn hơn. Hệ thống phụ thuộc vào một máy chủ trung tâm và lưu trạng thái trong bộ nhớ RAM, vì vậy chưa thể duy trì hoặc khôi phục trận đấu khi máy chủ gặp sự cố. Cơ chế khôi phục kết nối chỉ hoạt động khi máy khách còn giữ thông tin phiên, chưa hỗ trợ tiếp tục phiên chơi sau khi đóng hoàn toàn ứng dụng rồi mở lại. Bên cạnh đó, hệ thống chưa có chức năng tài khoản, lưu tiến trình và lịch sử thi đấu; các cơ chế xác thực, bảo mật truyền tải và chống gian lận cũng chưa được hoàn thiện.

Nhìn chung, kết quả cho thấy tính khả thi của việc xây dựng trò chơi multiplayer bằng Java và TCP, đồng thời tạo cơ sở để tiếp tục kiểm thử và hoàn thiện hệ thống trong tương lai.

VI. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Đề tài đã xây dựng Game Đặt Bomb Multiplayer bằng Java và Swing theo mô hình client–server sử dụng TCP. Hệ thống tích hợp phòng chơi, trò chuyện, di chuyển, bom, vật phẩm, mạng, hồi sinh và kết quả trận đấu. Thiết kế đặt quyền quyết định trạng thái tại máy chủ, kết hợp hàng đợi lệnh, snapshot full/delta và các định danh thao tác để hỗ trợ tính nhất quán giữa các máy khách.

Hồ sơ kiểm chứng ghi nhận hệ thống đã vượt qua toàn bộ 333 bài kiểm thử tự động, cùng chín kịch bản benchmark loopback với 25 dòng dữ liệu client đều đạt trạng thái PASS. Những kết quả này cung cấp bằng chứng vững chắc về tính đúng đắn của các đường xử lý cốt lõi, bao gồm cả các kịch bản tích hợp phức tạp dành riêng cho cơ chế khôi phục kết nối. Đồng thời, số liệu đo đạc cho phép đánh giá sơ bộ về lưu lượng và thời gian phản hồi. Dù cơ chế chịu lỗi đã được xác thực ở cấp độ mã nguồn, bước nghiệm thu thao tác gián đoạn mạng vật lý trên giao diện thực tế (LAN) vẫn là một hạng mục cần tiếp tục hoàn thiện.

Hướng phát triển tiếp theo của đề tài là nghiên cứu và bổ sung các cơ chế đồng bộ, xử lý mạng và quản lý trạng thái nhằm nâng cao tính ổn định, khả năng đáp ứng và khả năng mở rộng của hệ thống. Đồng thời, đề tài có thể tiếp tục hoàn thiện cơ chế phục hồi khi xảy ra gián đoạn kết nối, tăng cường bảo mật và bảo đảm tính công bằng giữa những người chơi. Các cơ chế này cần được kiểm chứng trong nhiều điều kiện vận hành khác nhau để đánh giá hiệu quả và khả năng áp dụng thực tế.

TÀI LIỆU THAM KHẢO

akien-mga. (n.d.). *DynaDungeons* [Mã nguồn]. GitHub. https://github.com/akien-mga/dynadungeons

arnobpl. (n.d.). *Bomberman Battle* [Mã nguồn]. GitHub. https://github.com/arnobpl/Bomberman-Battle

Eddy, W. (Ed.). (2022). *Transmission Control Protocol (TCP)* (RFC 9293). Internet Engineering Task Force. https://doi.org/10.17487/RFC9293

Fiedler, G. (2004, September 4). *Networked physics (2004)*. Gaffer On Games. https://gafferongames.com/post/networked_physics_2004/

Fiedler, G. (2014, November 30). *Snapshot interpolation*. Gaffer On Games. https://gafferongames.com/post/snapshot_interpolation/

Fiedler, G. (2015, January 4). *Snapshot compression*. Gaffer On Games. https://gafferongames.com/post/snapshot_compression/

Gambetta, G. (n.d.-a). *Client-server game architecture*. Fast-Paced Multiplayer. https://www.gabrielgambetta.com/client-server-game-architecture.html

Gambetta, G. (n.d.-b). *Client-side prediction and server reconciliation*. Fast-Paced Multiplayer. https://www.gabrielgambetta.com/client-side-prediction-server-reconciliation.html

Gambetta, G. (n.d.-c). *Entity interpolation*. Fast-Paced Multiplayer. https://www.gabrielgambetta.com/entity-interpolation.html

shuoros. (n.d.). *Bomberman* [Mã nguồn]. GitHub. https://github.com/shuoros/Bomberman

PHỤ LỤC

Phụ lục A Yêu cầu và tiêu chí kiểm chứng

Mã	Yêu cầu	Tiêu chí kiểm chứng
FR01	Kết nối và lobby	Client nhận danh sách phòng; create/join/leave hợp lệ
FR02	Ready và bắt đầu	Kiểm tra quyền host và điều kiện vào trận
FR03	Di chuyển và va chạm	Máy chủ xử lý ý định; không đi xuyên vật cản
FR04	Bom và nổ liên hoàn	Đặt đúng ô; kích nổ và phá block theo luật
FR05	Vật phẩm và mạng	Pickup một lần; chỉ số, khiên và hồi sinh đúng
FR06	Kết quả và chơi lại	Kết quả do server chốt; lượt mới có state mới
FR07	Khôi phục kết nối	Giữ chỗ; full resync; hết hạn xử lý đúng
NFR01	Nhất quán dữ liệu	Loại match cũ, event lặp và delta sai baseline
NFR02	Giới hạn tài nguyên	Queue/rate hữu hạn; xử lý peer không đọc
NFR03	Có thể tái lập	Script chạy, metadata, log và CSV được lưu

Bảng A1 Ma trận yêu cầu và bằng chứng cần đối chiếu

Các tiêu chí là căn cứ để đối chiếu, không phải toàn bộ đều đã được nghiệm thu trên LAN. FR01–FR06 và các đường xử lý nền tảng được hỗ trợ bởi hồ sơ kiểm thử lịch sử. FR07 cần đối chiếu bản cập nhật reconnect và bổ sung bằng chứng gián đoạn mạng thật.

Phụ lục B Kịch bản kiểm thử chức năng

Mã	Thao tác hoặc điều kiện đã kiểm tra	Kết quả cần xác nhận
IT01	Kiểm tra giới hạn phòng, quyền, ready/start, nhiều phòng, reconnect, replay và dọn runtime.	Trạng thái phòng nhất quán; start không tạo trận trùng; reconnect giữ phiên; tài nguyên được dọn đúng.
IT02	Kiểm tra quyền lobby, handshake, token, snapshot, resync và đặt bom.	Yêu cầu sai bị từ chối; kết quả trận, resync và đặt bom được server xác nhận đúng.
IT03	Reconnect và replay trong lobby có cấu hình.	Giữ runtime, cấu hình khi reconnect; replay tạo trận mới.
IT04	Kiểm tra roster, chuyển format và hết giờ.	Roster đúng; kết quả hòa nhất quán.
IT05	Recovery, replay và đổi cấu hình qua các format trận.	Chuyển trạng thái đúng, không sót dữ liệu giữa các trận.
IT06	Kiểm tra TCP, di chuyển, va chạm, giới hạn phòng, resync và shutdown.	Server áp dụng đúng collision và giới hạn; đồng bộ và dọn tài nguyên đúng.
IT07	Kiểm tra vật phẩm, sát thương, kết thúc trận, resync, hồi sinh và replay.	Kết quả và trạng thái trận nhất quán; không dùng lại state cũ.
IT08	Kiểm tra đặt bom, request trùng, reconnect và khởi tạo lobby.	Bom được đồng bộ; thao tác trùng không lặp; reconnect giữ bộ đếm đúng.
IT09	Kiểm tra gameplay khi offline, reconnect, token sai và hết thời gian giữ chỗ.	Khôi phục đúng trạng thái; trận đã kết thúc không sống lại; phiên hết hạn bị từ chối.
IT10	Kiểm tra reconnect, rewind, hết hạn, kết quả offline và replay ở chế độ Paradox.	Timeline và kết quả đúng; replay không giữ state cũ.

IT11	Kiểm tra đồng bộ timeline và bản tin đến trễ.	Bản tin cũ bị bỏ qua; đồng bộ tiếp tục trên timeline hiện tại.
IT12	Kiểm tra kết quả team offline, token, nhiều client recovery và hard drop.	Chỉ người mất kết nối bị loại; client còn lại tiếp tục đúng trạng thái.
IT13	Kiểm tra runtime lỗi, nhiều vòng trận và chu kỳ khởi động/dừng server.	Runtime và worker được dọn; không tích lũy listener qua các lần chạy.
IT14	Chạy launcher trong ba tiến trình Java riêng.	Các tiến trình giao tiếp và hoạt động độc lập.

Bảng B1 Các tình huống kiểm tra tích hợp

Bảng mô tả kết quả mong đợi để tái kiểm tra sản phẩm. Hồ sơ lịch sử ghi nhận các nhóm kiểm thử tương ứng, nhưng không thay cho biên bản thao tác GUI nhiều máy. Khi chạy nghiệm thu, ghi revision, môi trường, kết quả và đường dẫn log cho từng tình huống.

Phụ lục C Bảng Benchmark hiệu năng mạng

Kịch bản Test	Số Client	Độ trễ mạng (Ping P95 - ms)	Độ trễ di chuyển (Move ACK P95 - ms)	Độ trễ đặt bom (Plant ACK - ms)	Thời gian xử lý di chuyển (ms)	Thời gian xử lý đặt bom (ms)	Tốc độ truyền (Băng thông Gửi - B/s)	Tốc độ nhận (Băng thông Nhận - B/s)
Transport (Truyền tải)	1	4.129	-	-	-	-	94.99	94.99
Transport (Truyền tải)	2	8.315	-	-	-	-	95.00	95.00
Transport (Truyền tải)	4	4.357	-	-	-	-	94.99	94.99
Idle (Trạng thái rảnh)	2	3.787	-	-	-	-	95.00	486.99
Idle (Trạng thái rảnh)	4	3.547	-	-	-	-	94.98	583.70
Movement (Di chuyển)	2	18.084	64.114	-	49.493	-	1,728.91	6,430.74

Movement (Di chuyển)	4	17.120	76.549	-	61.477	-	1,749.57	9,868.00
Bombs (Đặt bom)	2	21.753	70.958	66.054	54.976	50.072	1,781.31	6,765.95
Bombs (Đặt bom)	4	17.685	77.531	49.161	62.589	34.219	1,789.33	10,288.42

Bảng C1 Bảng benchmark hiệu năng mạng

Bảng cho thấy Ping P95 và thời gian xác nhận di chuyển/đặt bom đều dưới 100 ms trong các lượt đo. Gameplay tạo lưu lượng mạng cao hơn rõ rệt so với Transport và Idle, đặc biệt ở chiều nhận.

Phụ lục D Kiểm tra khôi phục kết nối

Các tình huống sau được xây dựng theo hành vi hiện hành: giữ chỗ 30 giây kể từ khi máy chủ phát hiện mất kết nối; trận tiếp tục; nhân vật vẫn chịu sát thương; client chỉ mở input sau full .Đây là danh sách nghiệm thu cần thực hiện và đã qua kiểm thử.

Tình huống	Kết quả đã xác nhận
Ngắt rồi nối lại trước hạn	Giữ đúng phiên và match; đồng bộ đầy đủ trạng thái
Bom nổ trong lúc mất mạng	Đồng hồ và sát thương tiếp tục theo máy chủ
Trận kết thúc trong thời gian chờ	Nhận kết quả; không mở lại simulation cũ
Hết thời gian giữ chỗ	Chỉ loại người mất kết nối; trận xử lý tiếp theo luật
Token không hợp lệ	Không chiếm được phiên đang giữ chỗ
Socket cũ báo lỗi muộn	Không đóng hoặc xóa kết nối đã thay thế
Timer cũ chạy sau reconnect	Không loại nhầm người đã trở lại
Gửi tiếp MOVE và PLANT_BOMB	Sequence và requestId không gây lặp thao tác
Đóng hẳn ứng dụng	Không giả định tự khởi động lại hoặc lưu token
Chủ động Leave room	Áp dụng chính sách rời phòng, không coi là mất mạng

Bảng D1 Các trường hợp cần xác nhận cho reconnect

Các kết quả trên là kiểm thử tự động cục bộ, gồm kiểm thử logic phiên và kiểm thử tích hợp dùng kết nối TCP cục bộ. Chúng không phải kết quả đo trên LAN. Việc đóng hẳn ứng dụng, chủ động rời phòng và trải nghiệm khôi phục trên giao diện trong điều kiện mạng LAN không được suy ra là đã kiểm thử chỉ từ các kết quả này.